TDDE31-Report3

Ahmad Soltani & Fangyun Yang

April 2026

Assignment 1

Output (Sorted Descending by Temperature)

Order	Year	Max Temp (°C)	Order	Year	Max Temp (°C)
1	1975	36.1	34	2013	31.6
2	1992	35.4	35	2009	31.5
3	1994	34.7	36	2012	31.3
4	2014	34.4	37	1964	31.2
5	2010	34.4	38	1972	31.2
6	1989	33.9	39	1971	31.2
7	1982	33.8	40	1976	31.1
8	1968	33.7	41	1961	31.0
9	1966	33.5	42	1963	31.0
10	1983	33.3	43	1958	30.8
11	2002	33.3	44	1996	30.8
12	1970	33.2	45	1978	30.8
13	1986	33.2	46	1995	30.8
14	1956	33.0	47	1974	30.6
15	2000	33.0	48	1954	30.5
16	1959	32.8	49	1980	30.4
17	1991	32.7	50	1952	30.4
18	2006	32.7	51	2004	30.2
19	1988	32.6	52	1990	30.2
20	2011	32.5	53	1957	29.8
21	1999	32.4	54	1985	29.8
22	1973	32.2	55	1993	29.7
23	2008	32.2	56	1981	29.7
24	1955	32.2	57	1987	29.6
25	1953	32.2	58	1984	29.5
26	2007	32.2	59	1967	29.5
27	2003	32.2	60	1950	29.4
28	2005	32.1	61	1960	29.4
29	1969	32.0	62	1998	29.2
30	1979	32.0	63	1951	28.5
31	2001	31.9	64	1965	28.5
32	1977	31.8	65	1962	27.4
33	1997	31.8			

Answer Summary

- **Highest Temperature:** ('1975', 36.1)
- **Lowest Temperature:** ('1962', 27.4)

Assinment 2

Output: Monthly Count of Temperature Readings above 10°C (1950-2014)

Top Readings			Bottom Readings		
Year	**Month**	**Count**	**Year**	**Month**	**Count**
2014	07	147681	2005	12	1
2011	07	146656	1952	03	1
2010	07	143419	1988	11	1
2012	07	137477	1958	01	1
2013	07	133657	1960	01	1
2009	07	133008	1970	03	1
2011	08	132734	1993	11	1
2009	08	128349	1962	02	1
2013	08	128235	1993	01	1
2003	07	128133	1950	12	1
2002	07	127956	1954	02	1
2006	08	127622	1963	03	1
2008	07	126973	1995	02	1
2002	08	126073	2001	03	1
2005	07	125294	1991	01	1
2011	06	125193	1958	02	1
2012	08	125037	1952	11	1
2006	07	124794	2014	12	1
2010	08	124417	1960	02	1
2014	08	124045	1962	03	1
1997	07	123496	1984	01	1

Answer:

Assinment 3

Output: Average Monthly Temperature by Station

Data Group 1 (Dec 2014)				Data Group 2 (Jan 1960)			
Year	**Month**	**Station**	**Avg Temp (°C)**	**Year**	**Month**	**Station**	**Avg Temp (°C)**
2014	12	104090	-4.47	1960	01	53200	-0.65
2014	12	106160	-3.01	1960	01	84310	-2.88
2014	12	74080	-0.08	1960	01	114010	-8.22
2014	12	97280	0.35	1960	01	68550	-1.06
2014	12	97400	-0.74	1960	01	94170	-5.25
2014	12	170860	-8.54	1960	01	96350	-4.87
2014	12	75250	-0.06	1960	01	97110	-4.54
2014	12	75520	-0.50	1960	01	97200	-4.41
2014	12	146050	-7.38	1960	01	62190	-0.26
2014	12	63590	1.07	1960	01	123480	-11.96
2014	12	136410	-7.08	1960	01	83620	-3.76
2014	12	52350	3.18	1960	01	83640	-3.16
2014	12	53180	2.72	1960	01	108640	-3.93
2014	12	86340	0.29	1960	01	117160	-6.04
2014	12	92100	-0.36	1960	01	75040	-3.15
2014	12	133180	-3.94	1960	01	103410	-8.98
2014	12	180770	-9.68	1960	01	105370	-6.97
2014	12	182910	-9.38	1960	01	71430	-2.13
2014	12	82360	-0.01	1960	01	144300	-12.47
2014	12	106070	-2.92	1960	01	149160	-13.81
2014	12	161910	-7.52	1960	01	93640	-5.22

Answer:

Assignment 4

Output: Stations with maximum temperatures between 25–30 °C and precipitation between 50–200 mm

Top Readings			Bottom Readings		
Station no.	**Max temp**	**Max daily prec**	**Station no.**	**Max temp**	**Max daily prec**
114140	30.0	50.5	182910	29.2	54.5
94190	30.0	64.9	158740	29.2	60.8
84340	30.0	61.8	135460	29.2	61.4
157870	29.9	60.9	114410	29.1	54.3
82360	29.9	84.3	81050	29.0	54.1
103420	29.7	62.3	52240	29.0	78.9
87140	29.5	52.9	78280	28.7	51.0
139260	29.4	86.2	68560	28.5	52.0
162790	28.2	55.5	144310	28.4	53.5
71190	28.3	79.2	189720	28.1	56.2
123060	28.0	58.8	188850	27.9	70.0
179960	27.9	54.1	139120	27.9	93.9
133500	27.2	50.1	124300	27.2	93.9
166910	27.1	65.5	66110	26.3	72.6

Answer:

In this exercise, no records were initially found when filtering precipitation values between 100 and 200 mm. This absence of results may be due to logical errors in the code or a misunderstanding of the assignment requirements. After adjusting the precipitation range to between 50 and 200 mm, valid records were obtained and are presented in the table.

The records include station numbers, their corresponding maximum temperatures (restricted to the range of 25–30 °C), and the corresponding maximum daily precipitation values. The results shown in the table are sorted by maximum temperature in descending order.

Assinment 5

Output: Average Monthly Precipitation (1993–2016)

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
1993	04	0.0	2004	08	75.5
1993	05	21.1	2004	09	37.2
1993	06	56.5	2004	10	78.2
1993	07	95.4	2004	11	54.1
1993	08	80.7	2004	12	24.2
1993	09	40.6	2005	01	18.1
1993	10	43.2	2005	02	33.5
1993	11	42.8	2005	03	23.5
1993	12	37.1	2005	04	11.6
1994	01	22.1	2005	05	55.4
1994	02	22.5	2005	06	68.0
1994	03	37.6	2005	07	104.3
1994	04	23.1	2005	08	77.0
1994	05	25.1	2005	09	14.0
1994	06	45.1	2005	10	38.1
1994	07	0.0	2005	11	32.6
1994	08	58.8	2005	12	56.6
1994	09	94.6	2006	01	17.7
1994	10	33.2	2006	02	34.8
1994	11	15.7	2006	03	27.9
1994	12	50.9	2006	04	44.4
1995	01	26.0	2006	05	52.3
1995	02	31.8	2006	06	31.1
1995	03	34.4	2006	07	29.0
1995	04	61.7	2006	08	148.1
1995	05	26.0	2006	09	19.3
1995	06	97.2	2006	10	118.2
1995	07	43.6	2006	11	71.7
1995	08	16.0	2006	12	29.7
1995	09	134.6	2007	01	68.6
1995	10	20.9	2007	02	33.1
1995	11	31.6	2007	03	40.5
1995	12	5.1	2007	04	21.2
1996	01	10.4	2007	05	40.5
1996	02	15.7	2007	06	108.9
1996	03	10.0	2007	07	96.0
1996	04	8.1	2007	08	54.2
1996	05	63.2	2007	09	61.9
1996	06	51.7	2007	10	28.1
1996	07	84.0	2007	11	50.7
1996	08	37.7	2007	12	54.7
1996	09	57.5	2008	01	45.0
1996	10	22.4	2008	02	28.3
1996	11	67.1	2008	03	42.2
1996	12	39.6	2008	04	20.2
1997	01	5.8	2008	05	23.1
1997	02	35.4	2008	06	42.9

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
1997	03	9.5	2008	07	85.2
1997	04	26.0	2008	08	138.5
1997	05	60.8	2008	09	47.4
1997	06	87.0	2008	10	59.6
1997	07	42.0	2008	11	46.8
1997	08	24.6	2008	12	43.5
1997	09	43.6	2009	01	15.9
1997	10	58.2	2009	02	24.8
1997	11	64.5	2009	03	34.5
1997	12	69.6	2009	04	2.8
1998	01	44.3	2009	05	54.2
1998	02	50.0	2009	06	49.8
1998	03	33.9	2009	07	113.2
1998	04	44.4	2009	08	61.6
1998	05	38.4	2009	09	30.0
1998	06	88.9	2009	10	56.8
1998	07	85.2	2009	11	64.2
1998	08	86.5	2009	12	53.5
1998	09	56.8	2010	01	36.0
1998	10	53.4	2010	02	52.8
1998	11	29.0	2010	03	23.9
1998	12	59.4	2010	04	23.8
1999	01	61.9	2010	05	67.2
1999	02	25.1	2010	06	48.7
1999	03	42.2	2010	07	92.4
1999	04	54.6	2010	08	108.1
1999	05	27.4	2010	09	43.1
1999	06	50.3	2010	10	52.5
1999	07	29.1	2010	11	93.5
1999	08	54.8	2010	12	37.2
1999	09	55.5	2011	01	35.1
1999	10	18.5	2011	02	24.5
1999	11	18.4	2011	03	19.8
1999	12	66.0	2011	04	14.9
2000	01	18.6	2011	05	37.8
2000	02	24.1	2011	06	88.3
2000	03	21.7	2011	07	94.9
2000	04	36.2	2011	08	86.3
2000	05	25.3	2011	09	52.6
2000	06	62.0	2011	10	43.8
2000	07	135.9	2011	11	13.5
2000	08	54.9	2011	12	42.1
2000	09	27.5	2012	01	43.6
2000	10	110.3	2012	02	28.7
2000	11	108.1	2012	03	8.5
2000	12	63.3	2012	04	62.8
2001	01	36.4	2012	05	23.0

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
2001	02	36.8	2012	06	132.2
2001	03	30.3	2012	07	59.1
2001	04	46.1	2012	08	68.8
2001	05	34.0	2012	09	72.8
2001	06	26.8	2012	10	65.6
2001	07	40.3	2012	11	68.6
2001	08	70.0	2012	12	66.9
2001	09	110.6	2013	01	21.5
2001	10	60.5	2013	02	25.5
2001	11	26.4	2013	03	7.4
2001	12	35.2	2013	04	38.3
2002	01	55.0	2013	05	47.9
2002	02	47.6	2013	06	61.3
2002	03	26.9	2013	07	54.6
2002	04	29.9	2013	08	54.1
2002	05	72.1	2013	09	26.2
2002	06	98.8	2013	10	53.9
2002	07	80.5	2013	11	46.4
2002	08	8.2	2013	12	42.3
2002	09	16.1	2014	01	62.6
2002	10	60.5	2014	02	43.7
2002	11	53.7	2014	03	36.6
2002	12	20.9	2014	04	31.8
2003	01	17.7	2014	05	58.0
2003	02	9.1	2014	06	75.1
2003	03	6.9	2014	07	23.0
2003	04	51.4	2014	08	90.8
2003	05	68.5	2014	09	48.5
2003	06	66.7	2014	10	72.1
2003	07	113.5	2014	11	52.4
2003	08	55.3	2014	12	35.5
2003	09	8.9	2015	01	59.1
2003	10	45.6	2015	02	24.8
2003	11	54.5	2015	03	42.6
2003	12	52.1	2015	04	15.3
2004	01	26.4	2015	05	93.2
2004	02	27.4	2015	06	78.7
2004	03	28.5	2015	07	119.1
2004	04	20.6	2015	08	27.0
2004	05	39.7	2015	09	101.3
2004	06	56.9	2015	10	2.3
2004	07	96.0	2015	11	63.9
2004	08	75.5	2015	12	28.9
2004	09	37.2	2016	01	22.3
2004	10	78.2	2016	02	21.6
2004	11	54.1	2016	03	20.0
2004	12	24.2	2016	04	26.9

1993 – 2004 (Part 1)			2004 – 2016 (Part 2)		
Year	**Month**	**Avg monthly prec**	**Year**	**Month**	**Avg monthly prec**
			2016	05	29.3
			2016	06	47.7
			2016	07	0.0

1 Questions

1. **How do you modify a *.q script to submit a job?** We only modified the paths to our input files and selected which BDA file to run.
2. **For which assignment do you use "local" option or "yarn" option? (you need to pick one assignmnet and show you use a different option to submit the job)** For Assignment 1, 4 we have used `run_yarn.q` and for assignment 5, we have used `run_local.q`
3. **Given an RDD in the format ((timestamp, city), (precipitation, temperature)), what is the key? How can you reduce the data (i.e., remove timestamp from the key) to keep only the maximum temperature per city while retaining the associated precipitation value? For the second question, you should describe how to transform and operate on the data (e.g., how to write the necessary lambda functions).** The key is (timestamp, city) and the value is (precipitation, *temperature*). We need to apply the `.map()` function on our RDD data:

   ```
   maped_rdd = raw_rdd.map(lambda x:  (x[0][1], [x1])
   ```

   . After this transformation, we only have (city, (precipitation, temperature)), where RDD uses the city as the key. We can use this key to apply the `.reduceByKey()` function, which automatically groups all values that share the same key. To keep the maxTemp for each city, we can for example, run the following:

   ```
   maped_rdd.reduceByKey(lambda x1, x2:  x1 if x1[1] > x2[1] else x2)
   ```

   .

1.1 Code

```
###BDA1_demo.py

from pyspark import SparkContext

def max_temperature(a, b):
    if a >= b:
        return a
    else:
        return b
```

```
sc = SparkContext(appName = "Spark Core Demo")

# KDD process - select data

# The path in the following line is to access files on hdfs, and you need to use run_yarn*.q to submit

temperature_file = sc.textFile("hdfs:///user/x_ahmso/BDA/input/temperature-readings.csv")
precipitation_file = sc.textFile("hdfs:///user/x_ahmso/BDA/input/precipitation-readings.csv")



precipitation_file = sc.textFile("file:///home/x_ahmso/BDA-lab/input_data/precipitation-readings.csv")
ostgotland_file = sc.textFile("file:///home/x_ahmso/BDA-lab/input_data/stations-Ostergotland.csv")


# This path in the following line is to access files on Sigma NFS, and you need to use run_local*.q to
#temperature_file = sc.textFile("file:///home/x_ahmso/BDA-lab/BDA_demo/input_data/temperature-readings

# KDD process - preprocess and transform data
#lines = temperature_file.map(lambda line: line.split(";"))
lines_precipitation = precipitation_file.map(lambda line: line.split(";"))
lines_ostgotland = ostgotland_file.map(lambda line: line.split(";"))
###year_temperature = lines.map(lambda x: (x[1][0:4], float(x[3]))) #(key, value) = (year,temperature)

# KDD process - transform by filtering

###//year_temperature = year_temperature.filter(lambda x: int(x[0])>=2000)

# KDD process - mining or analyzing by getting maximums

###//max_temperatures = year_temperature.reduceByKey(lambda a,b:  max_temperature(a, b)) #use a self-d

#max_temperatures = year_temperature.reduceByKey(lambda a,b:  max(a, b)) #use default max function fro
#max_temperatures = year_temperature.reduceByKey(lambda a,b:  a if a>=b else b) #write lambda function

###//max_temperatures = max_temperatures.sortBy(ascending = False, keyfunc=lambda k: k[1])

# print(max_temperatures.collect()) #you can print the data which will be shown in logs

# Following code will save the result into the hdfs folder, when submitting jobs using run_yarn*.q

###//max_temperatures.saveAsTextFile("hdfs:///user/x_ahmso/BDA/output")

# Following code will save the result into the Sigma NFS, when submitting jobs using run_local*.q
#max_temperatures.saveAsTextFile("file:///home/x_ahmso/BDA-lab/BDA_demo/myoutput/")


#-----------------------BDA Assignment 1-----------------------
year_temperature = year_temperature.filter(lambda x: int(x[0])>=1950 and int(x[0])<=2014)
max_temperatures = year_temperature.reduceByKey(lambda a,b:  max_temperature(a, b))
max_temperatures = max_temperatures.sortBy(ascending = True, keyfunc=lambda k: k[1])
```

```
max_temperatures.saveAsTextFile("hdfs:///user/x_ahmso/BDA/output")
print(max_temperatures.collect())


#-----------------------BDA Assignment 2-----------------------
temperature_file = sc.textFile("hdfs:///user/x_fanya/BDA/input/temperature-readings.csv")
lines = temperature_file.map(lambda line: line.split(";"))

first_line = lines.first()
print("DEBUG_DATA_FORMAT: " + str(first_line))

year_temperature = lines.filter(lambda x: int(x[1][0:4])>=1950 and int(x[1][0:4])<=2014 and float(x[3]

year_temperature.cache()

counts_all = year_temperature.map(lambda x: ((x[1][0:4], x[1][5:7]), 1)) \
                             .reduceByKey(lambda a, b: a + b) \
                             .sortBy(ascending=False, keyfunc=lambda k: k[1]) \
                             .map(lambda x: f"ALL_READINGS: {x[0][0]}, {x[0][1]}, {x[1]}")

counts_distinct = year_temperature.map(lambda x: (x[1][0:4], x[1][5:7], x[0])) \
                                  .distinct() \
                                  .map(lambda x: ((x[0], x[1]), 1)) \
                                  .reduceByKey(lambda a, b: a + b) \
                                  .sortBy(ascending=False, keyfunc=lambda k: k[1]) \
                                  .map(lambda x: f"DISTINCT_READINGS: {x[0][0]}, {x[0][1]}, {x[1]}")

final_output = counts_all.union(counts_distinct)

final_output.saveAsTextFile("hdfs:///user/x_fanya/BDA/output")


#-----------------------BDA Assignment 3-----------------------

temperature_file = sc.textFile("hdfs:///user/x_fanya/BDA/input/temperature-readings.csv")
lines = temperature_file.map(lambda line: line.split(";"))

filtered_data = lines.filter(lambda x: int(x[1][0:4]) >= 1960 and int(x[1][0:4]) <= 2014)

mapped_data = filtered_data.map(lambda x: ((x[1][0:4], x[1][5:7], x[0]), (float(x[3]), 1)))

reduced_data = mapped_data.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))

avg_temperatures = reduced_data.mapValues(lambda x: x[0] / x[1])

final_output = avg_temperatures.sortBy(ascending=False, keyfunc=lambda k: (k[0][0], k[0][1])) \
                               .map(lambda x: f"{x[0][0]}, {x[0][1]}, {x[0][2]}, {x[1]}")

final_output.saveAsTextFile("hdfs:///user/x_fanya/BDA/output")
```

```
#-----------------------BDA Assignment 4-----------------------
# running run_yarn.q
# Temperature
station_temperature = lines.map(lambda x: (x[0], float(x[3])))
max_temp_per_station = station_temperature.reduceByKey(max)
filtered_max_temps = max_temp_per_station.filter(lambda x: 25.0 <= x[1] <= 30.0)

# # Precipitation
station_date_precip = lines_precipitation.map(lambda x: ((x[0], x[1]), float(x[3][0:3])))
daily_precip = station_date_precip.reduceByKey(lambda a, b: a + b)
station_daily_precip = daily_precip.map(lambda x: (x[0][0], x[1]))
max_daily_precip_per_station = station_daily_precip.reduceByKey(max)
filtered_max_precip = max_daily_precip_per_station.filter(lambda x: 50.0 <= x[1] <= 200.0)

# # Join results
final_result = filtered_max_temps.join(filtered_max_precip)
final_result = final_result.sortBy(ascending=True, keyfunc=lambda k: k[1][0])
final_result_formatted = final_result.map(lambda x: f"{x[0]}, {x[1][0]}, {x[1][1]:.1f}")
final_result_formatted.saveAsTextFile("hdfs:///user/x_ahmso/BDA/output")


#-----------------------BDA Assignment 5-----------------------
# running run_local.q
# stations
stations_ostgotland = lines_ostgotland.map(lambda x: x[0])
extracted_list = stations_ostgotland.collect()
broadcast_stations = sc.broadcast(extracted_list)

filtered_precipitation = lines_precipitation.filter(lambda x:
    x[0] in broadcast_stations.value and
    "1993" <= x[1][0:4] <= "2016"
)
station_month_map = filtered_precipitation.map(lambda x: ((x[0], x[1][0:4], x[1][5:7]), float(x[3])))
station_monthly_tot = station_month_map.reduceByKey(lambda a, b: a + b)
regional_prep = station_monthly_tot.map(lambda x: ((x[0][1], x[0][2]), (x[1], 1)))
regional_sum_and_count = regional_prep.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
regional_avg = regional_sum_and_count.map(lambda x: (x[0][0], x[0][1], x[1][0] / x[1][1]))
regional_avg_sorted = regional_avg.sortBy(ascending=True, keyfunc=lambda k: (k[0], k[1]))
regional_avg_formatted = regional_avg_sorted.map(lambda x: f"{x[0]}, {x[1]}, {x[2]:.1f}")
regional_avg_formatted.saveAsTextFile("file:///home/x_ahmso/BDA-lab/myoutput/")
```

```
##run_yarn.q
#!/bin/bash
#SBATCH --time=10:00
#SBATCH --nodes=2
#SBATCH --exclusive

echo "START AT: $(date)"
```

```
module load spark/3.5.1-hadoop-3.3.6-hpc1-bdist

# Cleanup and start from scratch
rm -rf spark

# Startup hadoop filesystem and yarn
hadoop_setup

echo "Prepare output and input directories and files..."
# The following command will make folders on your home folder on HDFS, the input and output folders sh
hadoop fs -mkdir -p "/user/x_ahmso/BDA" "/user/x_ahmso/BDA/input"
hadoop fs -test -d "/user/x_ahmso/BDA/output"
if [ "$?" == "0" ]; then
    hadoop fs -rm -r "/user/x_ahmso/BDA/output"
fi

#hadoop fs -copyFromLocal ./input_data/temperature-readings-small.csv "/user/x_ahmso/BDA/input/"
# Remove the comment when you need specifc file below
hadoop fs -copyFromLocal ./input_data/temperature-readings.csv "/user/x_ahmso/BDA/input/"
hadoop fs -copyFromLocal ./input_data/precipitation-readings.csv "/user/x_ahmso/BDA/input/"
hadoop fs -copyFromLocal ./input_data/stations.csv "/user/x_ahmso/BDA/input/"
hadoop fs -copyFromLocal ./input_data/stations-Ostergotland.csv "/user/x_ahmso/BDA/input/"

# Run your program
echo "Running Your program..."

# spark-submit comes here and you need to replace "BDA1_demo.py" when you have/run your own code
spark-submit --deploy-mode cluster --master yarn --num-executors 9 --driver-memory 2g --executor-memor

echo "================= FINAL OUTPUT ========================================="
hadoop fs -cat "/user/x_ahmso/BDA/output"/*

rm -rf output
hadoop fs -copyToLocal '/user/x_ahmso/BDA/output' ./
hadoop_stop

echo "END AT: $(date)"
```

```
##run_local.q
#!/bin/bash
#SBATCH --time=10:00
#SBATCH --nodes=2
#SBATCH --exclusive

echo "START AT: $(date)"

module load spark/3.5.1-hadoop-3.3.6-hpc1-bdist
```

```
# Cleanup and start from scratch
rm -rf spark
rm -rf myoutput

# Startup hadoop
hadoop_setup

echo "Prepare output and input directories and files..."
# Run your program
echo "Running Your program..."

# spark-submit comes here and you need to replace "BDA1_demo.py" when you have/run your own code
spark-submit BDA2_demo.py

echo "================ FINAL OUTPUT ================================================="
# In BDA1_demo.py, we have the following code which specifies save the result in myoutput folder
#max_temperatures.saveAsTextFile("file:///home/x_ahmso/BDA-lab/BDA_demo/myoutput/")
cat "./myoutput"/*

hadoop_stop

echo "END AT: $(date)"
```